Deep Learning for AI
Final Project Report

Student: Nguyen Hoang Minh - Student ID: 220078
Project: Stock Price Predictions

This report aims to document the process of completing the Final Project for the Deep Learning for Artificial Intelligence course at Fulbright University Vietnam.

Model V1

Summary

This model is the trivial approach to accomplish the “stock price prediction” problem. In essence, a common model architecture is designed to be trained separately with different companies’ data for their respective predictions.

This model architecture is a simple 1D CNN model, using 1D Convolution layers to extract the general shape of the price movement and Dense layers for the predictions. There is also the use of Batch Normalization and Dropout layers to reduce the chance of overfitting. Other than that, most of the effort was put in data engineering, to ensure the model can be trained on a quality dataset.

The goal of this model is to: utilize the most simple model architecture (1D CNN), and make the most out of the pre-existing data, without the need of domain knowledge.

Data Engineering

Data Cleanup

The given dataset was great - properly formatted data, no null samples, same format for all tables. However, there were dates that were missing entirely from tables, which makes it hard to create “X-days windows” of data for training. This is because the stock market is closed on the weekends, national holidays, or it is an error inherent to the dataset. For this version, we can safely ignore these days and only work on days that the stock market is open, which are the days present in the dataset.

Data Augmentation

Besides numerical data points of various price metrics and trade volume, the remaining question is how do we make use of the dates? The solution is to encode them into numerical values for the model to understand. For instance, we can convert the dates to an integer value of “days from epoch time” so the model understands the relationship of older and newer data. Another relationship that may be useful for the model’s learning is the periodic time features such as day of the week, day of the month and the month. These may hint at repeated patterns throughout history that the model can pick up.

Given dataset:

- Date
- Low
- Open
- Volume
- High
- Close
- Adjusted Close

Augmented dataset:

- Low
- High
- Open
- Close
- Volume
- Year
- Month
- Day
- Day of the week
- Days since Epoch

Data Normalization

While the data are now numbers, the nature of all of them are not suitable for a typical deep learning model. Our goal is to have every number preferably be inside the range of -1 to 1 or 0 to 1, and the relationship between the numbers should be meaningful (closer value results in closer numbers and vice versa).

For the information about the stock, they should be “decoupled” from the actual value, since we care about the movements and patterns, rather than the raw value. Therefore, they are converted into log-return form, which is ln(new/old).

For the time information, they can be separated into 2 types: the periodic features and the linear features. Linear features are “Year” and “Days since Epoch”. This can be normalized with min/max normalization, which we already have a good point for “min” value which is the Epoch time. The “max” value can be set arbitrarily, with the requirements that it is in the future so that all dataset value and future predicted value do not surpass 1.0, so here we can choose 2030 as the end point. Finally for the periodic time features, we use cyclical encoding to put them into a circle that preserves the relationship between points.

Augmented & normalized dataset:

- Low (log-return)
- High (log-return)
- Open (log-return)
- Close (log-return)
- Volume (log-return)
- Year (min-max normalized)
- Month (cyclical encoding)
- Day (cyclical encoding)
- Day of the week (cyclical encoding)
- Days since Epoch (min-max normalized)

Findings

While the dataset quality was great, between 1500+ files there were still errors such as duplicating fields or lines that messed up with the parser code. I had to manually go in and fix those errors. They are in LRCX.csv.

Model Engineering

Approach

For this version, a model is meant to be trained per company. To begin, companies with long-standing history were chosen to be the starting companies for model design and evaluation.

After experimenting with models, the approach chosen for this model is a ***return prediction classification model***.

Data Preprocessing

While there was a lot of work being done in Data Engineering, it was not good enough to be used for training. The main bottleneck was to let the model learn actual features instead of statistically aiming for the lowest loss.

Considering stock data is almost total noise, there is not much pattern for the model to find and depend on. From initial experiments, all models either underfit heavily by not giving out a definitive answer, to a total collapse where it is fixated on one particular answer, like how “a broken clock is correct twice a day”. These abnormal behaviors, while not popular among the course’s classwork and assignments, are what truly separates a model in a controlled environment and real-world conditions.

The first problem to solve is to stop the model from the 2 aforementioned pitfalls. If I really can do that, I would have a perfect stock price prediction model and I will be rich. That is not realistic at all. However, there is a weird but effective way to force the model out of one of the problems, and it has been in hindsight all along. A way to force a thoughtful answer out of a model is to turn a *prediction* model into a *classification* model. This is the first breaking change that totally switches the model behavior, and makes it hard to compare with other models from the class, who mostly keep the example’s min-max normalized data for value prediction.

Changing from a prediction model to a classification model is all about reducing the resolution of the prediction. Here I have designed the “buckets” to represent 5 states of the price return: Strong down (strong bearish), Down (bearish), Stationary (neutral), Up (bullish) and Strong up (very bullish). A practical way to map the price return values to these buckets is to divide the data equally into the buckets. However, I consider that as “cheating”, because it totally depends on the data distribution. After some manually tuning to find threshold values that mostly divide the data into equal buckets, I have the buckets of: -inf to -0.02, -0.02 to -0.005, -0.005 to 0.005, 0.005 to 0.02 and 0.02 to inf. These approximately translate to values of -2%, -0.5%, 0.5% and 2%.

The next step is to slice the “tables” to create the training samples. For this model, the model takes a 30-day window of multi-feature history data, and it needs to predict the next 7 days of single-feature price metric. Therefore, we need to generate a list of these “tables” for the model to use as training, validating and testing data later on.

Finally, with the inherently imbalanced dataset, the final step is to truncate the training data so that it has a balance distribution. However, since each sample has more than 1 data point, there is no way to get a totally flat distribution on the training data. The chosen solution is to only balance out the first-day prediction, therefore we accept that the training data for the 2nd to 7th day is imbalanced.

Model Architecture

As stated, this version has a very simple architecture. I used multiple 1D Convolution layers for the first half of the model, and the rest were Dense layers. This is a typical structure for a CNN model.

Result.

This model did not have good results. The model always collapses to the easiest choice. With the training undersampling changes, the first day result is decent but for the later days, the quality drops. The model size was big, but the training and inference speed was decent.

Model V1.1

Summary

This model differs from the previous one by using a more modern architecture. I settled on a LSTM-based architecture with hope that it is better than the V1 model.

Data Engineering

From what I saw, the “metadata” features of time were synthesized, so they fitted perfectly in the -1 to 1 range. Price metrics however fluctuated wildly within a much smaller range with an occasional outlier. Therefore, I scaled down the non-price metrics by 1000 times so it does not overwhelm the model. Also, this gave us a strategy of putting 0 in if in some situation, we do not have the metadata on hand, such as predicting in an unknown time era.

Model Engineering

With help from the Internet and AI-assisted design, I have a LSTM Seq2Seq model (Encoder-Decoder architecture) that uses several LSTM layers and TimeDistribution. This is a typical model for fixed-window time-series forecasting or classification, which is very helpful in this case.

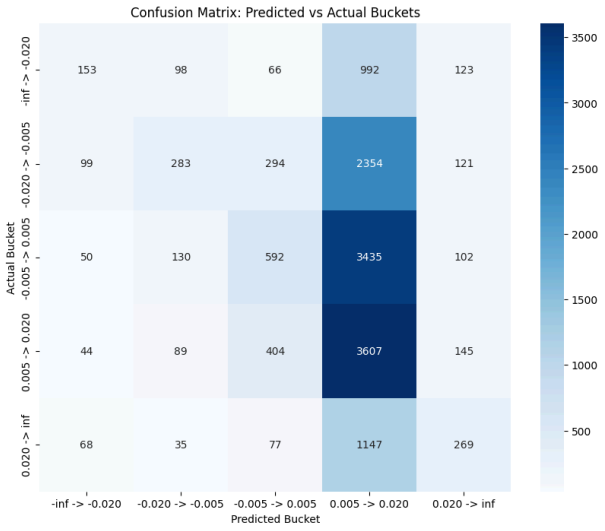
Result

This model was trained on NASDAQ-AAPL data. Apple as a company has a long-standing history, and the data is long enough to have patterns for the model to learn.

The model performed not very good on the 7-days forecast window. It reaches around 33% accuracy, which is only a bit better than pure guessing.

Classification Report:

	precision	recall	f1-score	support
-inf → -0.020	0.37	0.11	0.17	1432
-0.020 → -0.005	0.45	0.09	0.15	3151
-0.005 → 0.005	0.41	0.14	0.21	4309
0.005 → 0.020	0.31	0.84	0.46	4289
0.020 → inf	0.35	0.17	0.23	1596
accuracy			0.33	14777
macro avg	0.38	0.27	0.24	14777
weighted avg	0.38	0.33	0.27	14777

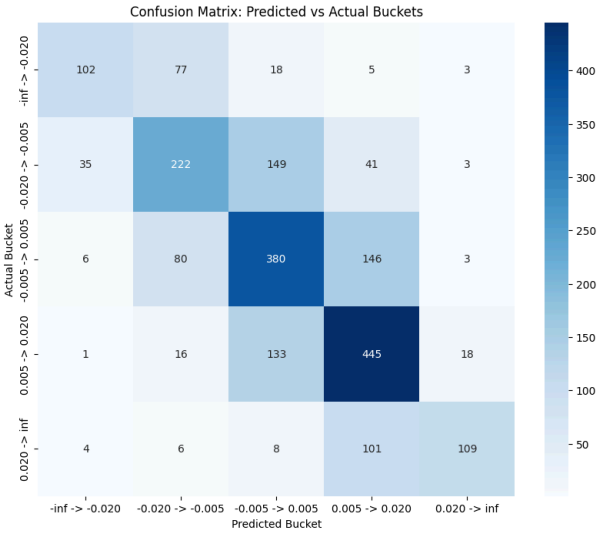


Confusion Matrix for the model 7-days performance

However, if we only look at the first day performance, the model did pretty well at ~60% accuracy.

Classification Report:

	precision	recall	f1-score	support
-inf → -0.020	0.69	0.50	0.58	205
-0.020 → -0.005	0.55	0.49	0.52	450
-0.005 → 0.005	0.55	0.62	0.58	615
0.005 → 0.020	0.60	0.73	0.66	613
0.020 → inf	0.80	0.48	0.60	228
accuracy			0.60	2111
macro avg	0.64	0.56	0.59	2111
weighted avg	0.61	0.60	0.59	2111



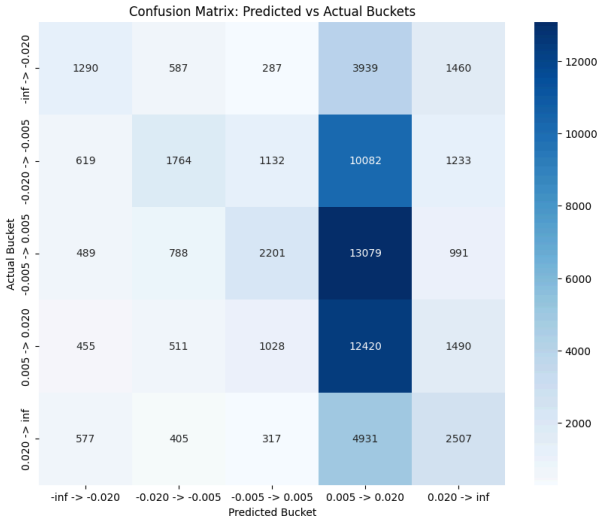
Confusion Matrix for the model 1-day performance

One benefit of a log-return model compared to a min-max normalized model is that the model does the prediction based on the price movement, not the price value itself. While the min-max normalization helps with the scaling, it is for one stock only, since other stocks need to be normalized separately with different scalars. That does not happen with log-return normalization because the movement is universal.

To verify this hypothesis, I used the NASDAQ-AAPL model to try to predict NASDAQ-MSFT data, and VNINDEX-ACB data as well.

This is the 7-days performance for NASDAQ-MSFT.

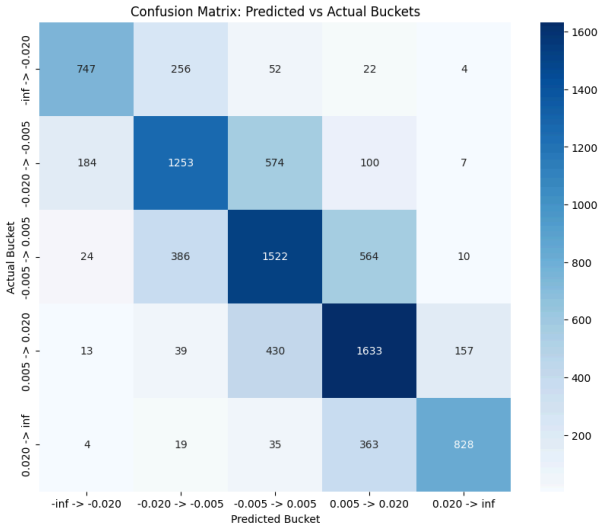
```
Classification Report:
                   precision    recall  f1-score   support

  -inf → -0.020       0.38      0.17      0.23      7563
-0.020 → -0.005       0.44      0.12      0.19     14830
 -0.005 → 0.005       0.44      0.13      0.20     17548
  0.005 → 0.020       0.28      0.78      0.41     15904
    0.020 → inf       0.33      0.29      0.31      8737

      accuracy                            0.31     64582
     macro avg       0.37      0.30      0.27     64582
  weighted avg       0.38      0.31      0.27     64582
```



Confusion Matrix for the model 7-days performance

This is the 1-day performance for NASDAQ-MSFT.

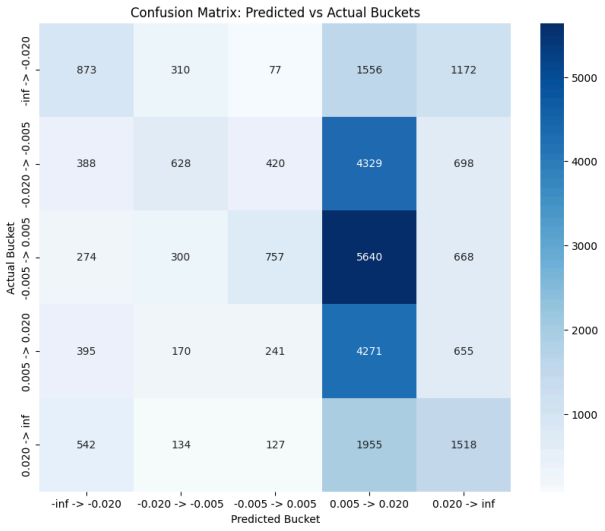
Classification Report:				
	precision	recall	f1-score	support
-inf → -0.020	0.77	0.69	0.73	1081
-0.020 → -0.005	0.64	0.59	0.62	2118
-0.005 → 0.005	0.58	0.61	0.59	2506
0.005 → 0.020	0.61	0.72	0.66	2272
0.020 → inf	0.82	0.66	0.73	1249
accuracy			0.65	9226
macro avg	0.68	0.65	0.67	9226
weighted avg	0.66	0.65	0.65	9226



Confusion Matrix for the model 1-day performance

This is the 7-days performance for VNINDEX-ACB.

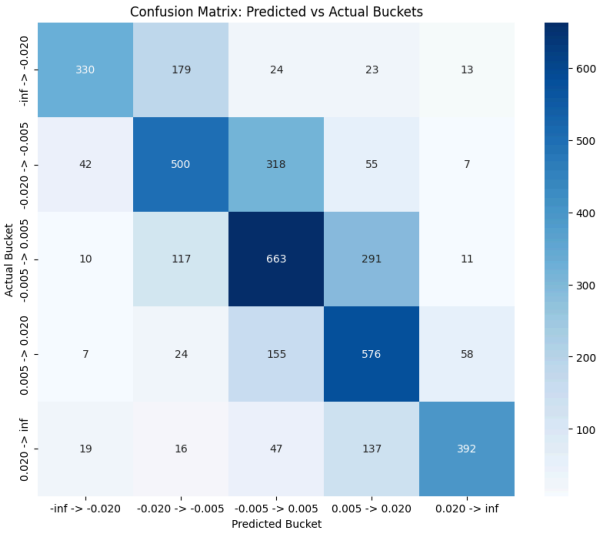
```
Classification Report:
                    precision    recall  f1-score   support
  -inf → -0.020          0.35      0.22      0.27      3988
-0.020 → -0.005          0.41      0.10      0.16      6463
 -0.005 → 0.005          0.47      0.10      0.16      7639
  0.005 → 0.020          0.24      0.75      0.36      5732
    0.020 → inf          0.32      0.36      0.34      4276
       accuracy                             0.29     28098
      macro avg          0.36      0.30      0.26     28098
   weighted avg          0.37      0.29      0.24     28098
```



Confusion Matrix for the model 7-days performance

This is the 1-day performance for VNINDEX-ACB.

```
Classification Report:
                    precision    recall  f1-score   support
  -inf → -0.020          0.81      0.58      0.68       569
-0.020 → -0.005          0.60      0.54      0.57       922
 -0.005 → 0.005          0.55      0.61      0.58      1092
  0.005 → 0.020          0.53      0.70      0.61       820
    0.020 → inf          0.81      0.64      0.72       611
       accuracy                             0.61      4014
      macro avg          0.66      0.61      0.63      4014
   weighted avg          0.63      0.61      0.62      4014
```



Confusion Matrix for the model 1-day performance

To my surprises, the model performed similarly on the new datasets it has never seen before. This gave me the confidence that a log-return based model, while it does not have a high accuracy number, can actually learn something from the stock price data.

Compared to the CNN model (V1), the model has better accuracy while being a lot smaller, thanks to the lack of Dense layers. It also ran as fast or faster than the V1 model.

A disclaimer on the model's performance

Summary

While my models with its unique design choice (log-return normalization versus min-max normalization, classification versus prediction) has underwhelming numbers, I want to clarify why I am proud of it and why I would not switch back to a more traditional design.

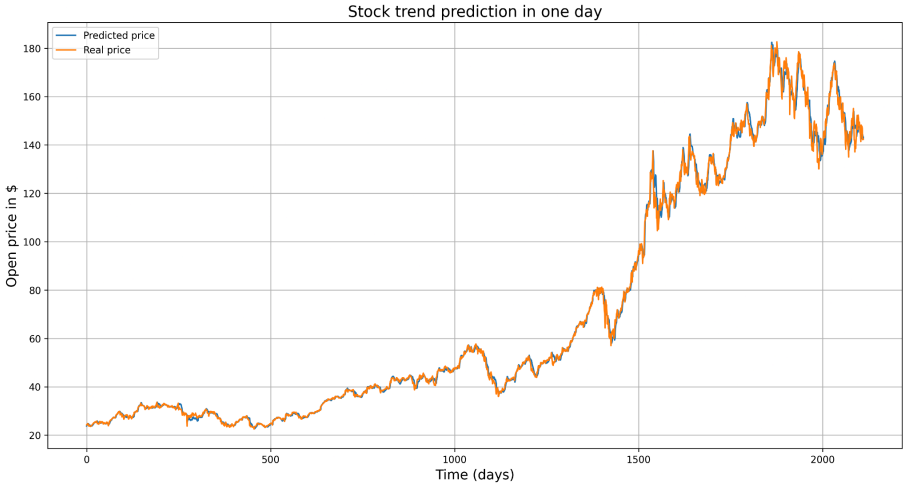
Analysis

Stock price behavior

If it really is that easy to "learn a Deep Learning class" and "get rich by trading with the prediction model", everyone would be rich now, especially the quantitative trading firms. Stock price is even considered as "almost noise" by every math course out there, as its fluctuations are wild and chaotic, almost "random" by design.

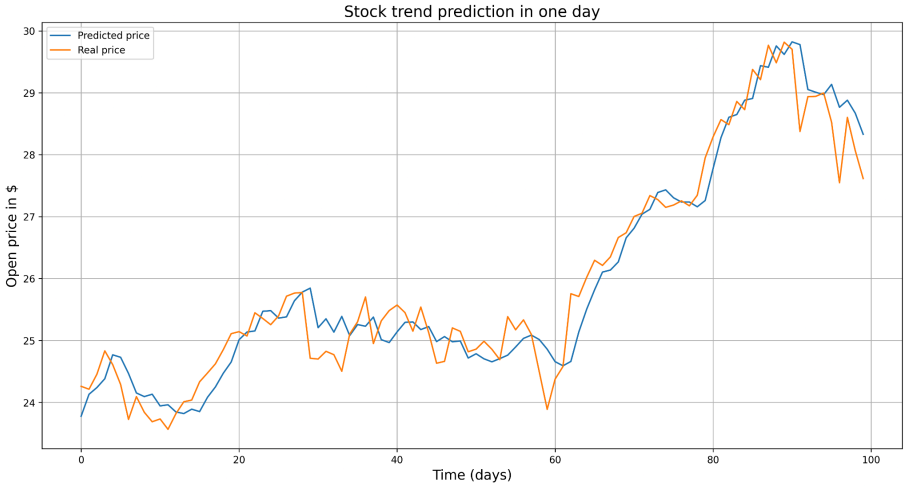
Criticism of the Min-max Normalization method

The pitfall of these stock prediction models is that it is easy to visually see the numbers going up and down. However, without careful consideration, a model which visually looks good can be terrible for actual use.



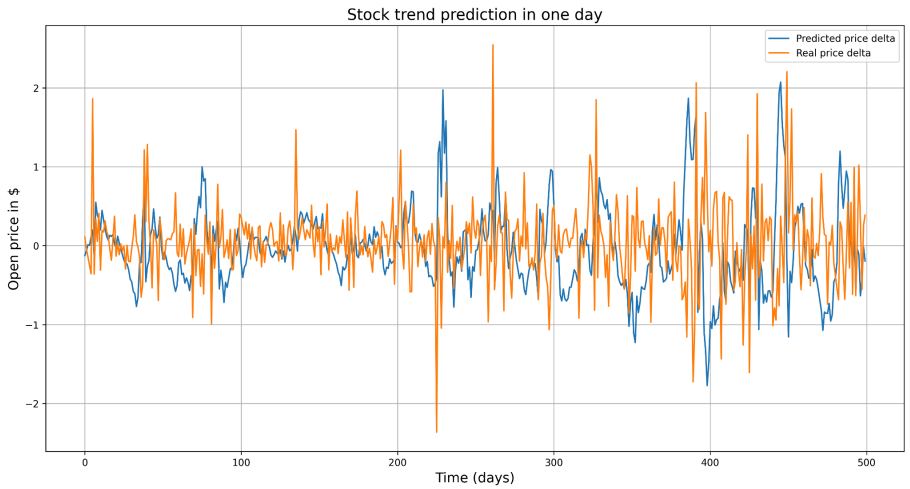
Stock price prediction chart in the provided sample code

This is the result of the model introduced in the sample code. Visually, it looks pretty good, as the two lines almost coincide. However, the trouble comes when we zoom in closer.



Stock price prediction chart from the provided model, day 1-100

As we can see, the blue line closely resembles the orange line, but always 1 day later. We can see that the model is “predicting” by just returning the last day result, which is inside the input data. This is the behavior of minimizing loss by returning the safest answer: the stock does not change at all. The chart may show that the prediction is almost perfect, but in reality, the model is a people-pleaser and we cannot use its signal to do trades. This is even more clear if we look at the price movements.



Stock price prediction delta chart from the provided model, day 500-1000

It is clear now that the price movements are almost “noise” and the model is just returning random fluctuations.

Even though this is an “example” model in the sample code, the same phenomenon will probably happen to almost everyone in class. I believe that without careful consideration, these nice-looking charts will fool anyone thinking that a model can easily predict the stock prices.

Application

Summary

This model only returns the price prediction. To use it for actual trades, we need to develop our trading strategies.

Understanding the strength and weakness of our model is the key to getting the right strategy.

Model analysis

Since our model is a “trading” model, rather than an “investing” model, we will design our strategy to act as a day-trader who buys and sells stocks based on technical factors, rather than buying for an investment to sell in the long future.

And from our results, the model can only predict with decent quality for the first few days, so we will build our strategies mainly from the 1st day result. It may not lead to the best returns since there’s only so much movement a stock can have in a day, but from our findings, we cannot predict with good certainty far into the future.

Buying/selling signal

I want to design the buy/sell signal based on the fact that we want to make profit the next day.

Therefore, to have our signal, if the model predicts that the price will go down, we sell now and buy back in 1 day after. Conversely, if the model predicts that the price will go up, we buy now and sell tomorrow.

Portfolio selection

Profitable stock selection

Because our model is a trading model, it does not care about whether a company is doing well or not, we are only making decisions to get our money. Therefore, every stock can be profitable if we are smart about it. If something is going to increase, we place a long call. If something is going to decrease, we place a long put.

Risk management

With that said, we should choose the companies that we do our transactions in. My strategy for this problem is simple. We run our model on companies past data, and choose the top companies with the best accuracy scores as our “confident” companies. These are the companies that our model is correct most often, so the risk is lowest when we use our model on them.

Portfolio composition

I do not want to sound lazy but this is not what our model is meant to help with. Our model is meant for trading, not investing, and without an investment there is nothing to build our portfolio.

Deployment

Model deployment

The model is saved to a .keras file, put on a server and served via a FastAPI webserver. This webserver has a “/predict” endpoint that receives the input (normalized) data and returns the prediction from the model (raw).

FastAPI was chosen as it was one of the Python web libraries, and hosting an API with it was the easiest compared to Flask or Django.

The reason I used a webserver library instead of a “tool” meant for the job is so that we have more freedom on designing our system. While it required more code written, code had become incredibly cheap to write thanks to generative AI. An example of the freedom we have here is that there is rate-limitting implemented for this model, so that we do not overwhelm the server by multiple model instances running at once, especially since this is a low-powered CPU-only server.

SaaS interface

To serve our model, I decided to have AI write a webpage to interact with the model to let the user try the price prediction model, and see the actual price movement.

My decision to use web technologies for the website instead of tools like Streamlit is to use the right tool for the right job. Tools like Streamlit or BI tools are meant for prototyping or internal use with a quick web interface, and not as robust as actual web with code written.

Automation workflow

To act as the core of our system, I have PocketBase running as our backend. It has many internal tools, but here we are using 2 of them: a SQL-based database, and a job scheduling engine. The database is used to store the stock price metrics, and the scheduling engine is running a CRON job to periodically check the Yahoo Finance API and pull the latest stock data back to the database. This database is integrated with the webpage, so users do not have to manually put in numbers.

Statement of AI use

For this project, I consider AI as a good friend with a lot of knowledge I can learn from. Therefore, there were a lot of AI-written snippets of code throughout the project. I only used a free-tier AI chatbot, not AI agents or AI-powered IDEs.

Especially considering I have not learned Machine Learning nor Data Visualization, there were a lot of concepts I did not know. Getting help from AI has been crucial to answering my questions and helping me write the code snippets since I am very unfamiliar with tools like Keras or SKlearn.

The ideas for everything came from me, and I only asked AI to write the code to realize that idea.

Conclusion

This project showed me how a Deep Learning model may be ideated, designed, used and deployed. It definitely was cool to have the complete pipeline done by myself. The only disappointing bit is the model did not perform very well, but other than that everything worked flawlessly. Things are modular, so that if I want a better model, it is a drop-in replacement without much trouble.

Most of my time was spent working with the handicap of dealing with log-return normalized numbers metrics for the price prediction model, so I did not have much time for the other indicators. Also, for the Extra credits, I spent more time than others to have a sustainable tech stack for an industry-standard system, not just prototyping tools bundled together for a short demo. If this system is expanded, I have much more confidence that my system scales better than others, even if right now it does not look as impressive.